

Lemuel 프로젝트에서 쓰이는 자료구조

이커머스 + 정산 MSA 플랫폼(Java 25 / Spring Boot 4)에서 **실제로 사용 중인** 자료구조를 코드 근거와 함께 정리한 문서. 분류는 (1) Java Collections Framework 실사용 → (2) 동시성 자료구조 → (3) 도메인·개념적 자료구조 → (4) 캐시 자료구조 → (5) 영속(DB-backed) 큐 순.

조사 기준: `**/src/main/java/**/*.java` 의 컬렉션 타입 사용 134건(46개 파일) + 핵심 도메인/인프라 클래스 정독.

1) Java Collections Framework 실사용

자료구조	주 용도	대표 사용처
<code>ArrayList</code> / <code>List</code>	도메인 컬렉션, 조회 결과, DTO 목록	전반 (Cart 아이템, Order 라인, 불일치 목록 등)
<code>HashMap</code>	키 기반 인덱싱·집계·카운팅	<code>PgReconciliationMatcher</code> (대사 매칭)
<code>LinkedHashMap</code>	삽입 순서가 의미 있는 맵 (우선순위 보존)	<code>PgRouter.adaptersByProvider</code> (PG fallback 체인 순서)
<code>HashSet</code>	중복 검출, 멤버십 체크	<code>PgReconciliationMatcher.pgDuplicates</code>
<code>EnumMap</code> 성격의 <code>Map<Enum, ></code>	PG provider → 어댑터/health 매핑	<code>PgRouter.healthSnapshot()</code>
<code>record</code>	불변 값 객체 (결과/행 표현)	<code>MatchResult</code> , <code>PgTransactionRow</code> , <code>InternalPaymentRow</code>

핵심 사례 — 양방향 해시 매칭 (`PgReconciliationMatcher`)

PG 정산파일 vs 내부 결제원장 대사 알고리즘. `pg_transaction_id` 를 키로:

- `Map<String, Integer>` + `merge(k, 1, Integer::sum)` → 중복 거래 카운팅
- `Map<String, Row>` + `putIfAbsent` → 키 기반 인덱스($O(1)$ 조회)로 $O(n^2)$ 중첩 루프 회피
- `Set<String>` → 중복 키 집합
- 양방향 스캔(PG→내부, 내부→PG)으로 `MISSING_INTERNAL` / `MISSING_PG` / `AMOUNT_MISMATCH` / `ROUNDING_DIFF` / `DUPLICATE` 분류
- 프레임워크 의존성 0 (순수 도메인 로직)

2) 동시성(Concurrency) 자료구조

자료구조 / 기법	용도	사용처
<code>ConcurrentHashMap</code>	lock-free 동시 맵 — 키별 lazy 생성	<code>RateLimitRegistry.buckets</code>
<code>computeIfAbsent</code>	원자적 lazy 초기화	토큰 버킷·캐시 L1 생성

자료구조 / 기법	용도	사용처
@Version (낙관적 락)	동시 확정/지급 충돌 방어	settlements.version, payouts.version
Pessimistic Lock + Idempotency-Key	환불 동시성 방어	refunds.idempotency_key
AtomicInteger/Long/Reference	카운터-플래그	스케줄러-메트릭 계열

핵심 사례 — 토큰 버킷 레지스트리 (RateLimitRegistry)

- Map<String, Bucket> = new ConcurrentHashMap<>() 에 (정책명|키) 합성 키로 버킷 보관.
- computeIfAbsent 로 최초 요청 시에만 버킷 생성(스레드 안전). 알고리즘 자체는 Bucket4j **Token Bucket**(Bandwidth capacity + greedy refill).

3) 도메인·개념적 자료구조

3-1. 트리(Tree) — 카테고리 계층

- EcommerceCategory / Category: parentId 셀프 참조 + List<EcommerceCategory> children 로 표현하는 **N-ary 트리**.
- 깊이 제한 MAX_DEPTH = 2(0/1/2 3단계), validateParentId() 로 순환 참조 방지, changeParent() 시 깊이 재검증.
- DB: categories.parent_id, ecommerce_categories.parent_id 인접 리스트(Adjacency List) 모델.

3-2. 상태 기계(State Machine) — enum 전이

도메인 상태는 enum + 허용 전이 규칙으로 모델링된 유한 상태 기계(FSM):

- Payment: READY → AUTHORIZED → CAPTURED → REFUNDED
- Settlement: REQUESTED → PROCESSING → DONE / FAILED / CANCELED
- Order: CREATED → PAID → REFUNDED / CANCELED
- Payout: REQUESTED → SENDING → COMPLETED / FAILED / CANCELED
- Chargeback: OPEN → ACCEPTED / REJECTED
- Ledger: PENDING → POSTED → REVERSED

3-3. 복식부기 원장(Double-Entry Ledger)

- ledger_entries: debit_account(차변) / credit_account(대변) / amount 쌍으로 회계 항등식 유지.
- reference_type + reference_id 로 정산/환불 원천 참조, 역분개(ReverseEntryService)로 정정.

3-4. 라우팅 테이블 + Fallback 체인 (PgRouter)

- LinkedHashMap<PaymentGateway, Adapter> — 순서가 곧 우선순위.
- 선택 우선순위: 고객 거래 우선 PG → 결제수단별 1순위 → fallback 체인 순회. unhealthy 시 다음 후보로 자동 전환.
- 거래 ID prefix 로 원처리 PG 역추적(resolveByTransactionId).

3-5. 복합 결제(Split Payment) — 1:N 분할

- **payments** 1건 : **payment_tenders** N건. 카드+포인트 등 여러 결제수단을 분할 합산.

3-6. 멱등 집합(Idempotency Set)

- 3단 방어: **outbox_events.event_id**(UUID UNIQUE) → **processed_events**(consumer_group+event_id 복합 PK) → **settlements.payment_id**(UNIQUE). 분산 환경의 "이미 처리된 이벤트 집합" 표현.

4) 캐시(Cache) 자료구조

계층	자료구조	사용처
L1 (로컬)	Caffeine Cache (Window-TinyLFU 제거 정책)	CacheConfig, TwoTierCacheManager
L2 (공유)	Redis	TwoTierCache (opt-in)
무효화	Pub/Sub 발행	CacheInvalidationPublisher — 인스턴스 로컬 L1 무효화 전파

- 기본은 로컬 전용 Caffeine. opt-in 시 **2-tier(L1 Caffeine + L2 Redis)** 구성, 각 캐시가 전용 L1 을 갖고 L2-무효화 발행기를 공유.

5) 영속(DB-backed) 큐 / 비동기 버퍼

메모리 큐가 아니라 **테이블로 구현한 큐** — 재기동·다중 워커에도 유실 없는 작업 버퍼.

큐	테이블	역할
Transactional Outbox	outbox_events	도메인 이벤트 → Kafka 발행 버퍼 (claim 기반 다중 워커 배치 발행)
Ledger Outbox	ledger_outbox	원장 이벤트 비동기 발행
ES 색인 큐	settlement_index_queue	정산 변경 → Elasticsearch 색인 작업 큐
DLQ / 재처리	Kafka DLT	DlqReplayService — 소비 실패 메시지 재처리

Outbox 는 **claim**(SELECT ... FOR UPDATE SKIP LOCKED 류) 기반으로 여러 poller 워커가 경합 없이 배치 발행 — 사실상 **분산 작업 큐(work queue)** 패턴.

정리

- **실사용 컬렉션**: **List/ArrayList, HashMap, LinkedHashMap**(순서 보존), **HashSet, ConcurrentHashMap**(동시성), **record**(불변).
- **알고리즘적 핵심**: 해시 인덱스 기반 양방향 대사 매칭(**PgReconciliationMatcher**), Token Bucket Rate Limit.
- **개념적 자료구조**: N-ary 카테고리 트리(순환 방지·깊이 제한), enum 유한 상태 기계, 복식부기 원장, PG 라우팅 테이블+fallback 체인, 멱등 집합, DB-backed 분산 큐.
- **캐시**: Caffeine(L1) + Redis(L2) 2-tier + Pub/Sub 무효화.

부록) 백엔드 개발자가 반드시 알아야 할 자료구조

위 1~5장이 "이 프로젝트가 쓰는 것"이라면, 이 부록은 **백엔드 엔지니어라면 원리·복잡도·트레이드오프를 설명할 수 있어야 하는** 핵심 자료구조를 정리한다. 각 항목은 *언제 쓰나* → *복잡도* → *백엔드에서의 실제 등장 위치* 순.

A) Big-O 먼저

표기	의미	예
$O(1)$	입력 크기와 무관	해시 조회, 배열 인덱싱
$O(\log n)$	매 단계 절반으로	균형트리·B+Tree 탐색, 이진탐색
$O(n)$	선형 스캔	정렬 안 된 목록 검색, 인덱스 없는 풀스캔
$O(n \log n)$	비교 정렬의 하한	머지/퀵/힙 정렬, ORDER BY
$O(n^2)$	중첩 루프	나이브 대사 매칭(→ 해시로 $O(n)$ 회피)

시간만큼 **공간 복잡도와 캐시 지역성(locality)** 도 본다. 배열은 연속 메모리라 같은 O 라도 링크드 리스트보다 빠른 경우가 많다.

B) 선형 자료구조

구조	조회	삽입/삭제	핵심 특성	백엔드 등장
배열 / 동적배열 (ArrayList)	$O(1)$ 인덱스	끝 amortized $O(1)$, 중간 $O(n)$	연속 메모리·캐시 친화	DTO 목록, 페이지 결과
링크드 리스트	$O(n)$	노드 핸들 있으면 $O(1)$	포인터 추적·캐시 비친화	LRU의 이중연결리스트, Deque 내부
스택 (LIFO)	—	push/pop $O(1)$	되돌리기·재귀·DFS	호출 스택, 트랜잭션 롤백 경계, 식 파싱
큐 (FIFO)	—	enqueue/dequeue $O(1)$	순서 보존 처리	작업 큐, BFS, 요청 버퍼
덱(Deque)	—	양끝 $O(1)$	스택+큐	슬라이딩 윈도우, 워크스틸 링 풀
링 버퍼(순환큐)	—	$O(1)$	고정 크기·덮어쓰기	로그 버퍼, 메트릭 윈도우, Disruptor

큐는 백엔드의 심장: 메모리 큐(**ArrayBlockingQueue**)부터 분산 큐(Kafka), DB-backed 큐(5장 Outbox)까지 동일한 FIFO 개념의 스케일업이다.

C) 해시 테이블 — 가장 많이 쓰는 구조

- **원리**: **$\text{hash(key)} \% \text{buckets}$** 로 버킷 배정 → 평균 $O(1)$ 조회/삽입.
- **충돌 처리**: 체이닝(버킷마다 리스트/트리 — Java **HashMap**은 버킷이 길어지면 트리화하여 최악 $O(\log n)$ 보장) vs 오픈 어드레싱(선형/이중 해싱).
- **load factor**(0.75 등) 초과 시 **리사이즈(reshape)** — 그래서 크기를 미리 알면 초기 용량 지정이 유리.
- **순서**: 기본 **HashMap**은 순서 없음 → 순서 필요 시 **LinkedHashMap**, 정렬 필요 시 **TreeMap**($O(\log n)$).

백엔드 응용	설명
인메모리 인덱스/캐시	<code>Map<id, entity></code> O(1) 조회 (이 프로젝트 대사 매치)
일관성 해싱(Consistent Hashing)	샤딩/분산 캐시에서 노드 추가·제거 시 재배포 최소화 — 링 위에 노드·키를 해싱, 가상 노드로 균형
파티셔닝	Kafka 파티션 키, DB 샤드 키
역등 집합	<code>HashSet</code> 으로 "이미 처리됨" 체크

D) 트리 — 정렬·범위·계층

구조	탐색	특성	백엔드 등장
이진탐색트리 (BST)	평균 $O(\log n)$	편향 시 $O(n)$ 로 퇴화	개념 기반
균형트리(AVL / Red-Black)	$O(\log n)$ 보장	회전으로 균형 유지	<code>TreeMap/TreeSet</code> , 리눅스 스케줄러, Java <code>HashMap</code> 버킷 트리화
B-Tree / B+ Tree	$O(\log n)$	노드가 넓음(수백 키) → 디스크 I/O 최소화, B+ Tree는 리프가 연결리스트라 범위 스캔 강함	관계형 DB 인덱스의 표준 (PostgreSQL 기본 인덱스)
힙(Heap) / 우선순위 큐	top $O(1)$, 삽입/삭제 $O(\log n)$	부분 정렬(완전이진트리)	작업 스케줄러, 타임아웃 관리, Dijkstra, Top-K
트라이(Trie)	$O(\text{키 길이})$	접두사 공유	자동완성, IP 라우팅 테이블, URL 라우터

B+ Tree ↔ 이 프로젝트: `V20260611100000` 에서 추가한 DB 인덱스가 바로 B+ Tree다. 인덱스 없는 컬럼 조회 = $O(n)$ 풀스캔(Seq Scan), 인덱스 있으면 $O(\log n)$ 탐색 + 리프 범위 스캔. 복합 인덱스 `(a, b)` 가 `WHERE a=? ORDER BY b` 를 정렬 없이 처리하는 이유도 B+ Tree 리프가 (a,b) 순으로 정렬·연결돼 있기 때문.

힙 ↔ 우선순위 큐: `PriorityQueue`(Java)는 배열 기반 이진 힙. "가장 임박한 마감/최고 우선순위"를 $O(1)$ 에 꺼내야 하는 스케줄링·레이트리밋 리필·재시도 백오프에 쓰인다.

E) 그래프

- **표현**: 인접 리스트(`Map<Node, List<Node>>`, 희소 그래프에 유리) vs 인접 행렬(밀집·간선 존재 $O(1)$ 확인).
- **순회**: BFS(최단 경로/레벨), DFS(연결성/사이클/위상).
- **위상 정렬(Topological Sort)**: 의존성 순서화 — 빌드 시스템, 작업 DAG, 마이그레이션 순서, 스프링 빈 의존성.
- **최단 경로**: Dijkstra(가중치 ≥ 0 , 힙 사용 $O(E \log V)$) — 배달/물류·네트워크 라우팅.
- **사이클 탐지**: 외래키 순환·카테고리 트리 순환 참조 방지(3-1장의 `validateParentId`)가 사실상 그래프 사이클 체크.

F) 집합 · 확률적(probabilistic) 자료구조

대용량에서 정확도를 약간 포기하고 메모리·속도를 얻는 구조 — 백엔드 면접 단골.

구조	용도	트레이드오프
Set / BitSet	멤버십-플래그 집합	정확하지만 원소 수만큼 메모리
Bloom Filter	"없음"을 빠르게 단정(있음은 false positive 가능)	캐시 penetration 방지, DB 조회 전 필터, LSM 존재 확인
HyperLogLog	고유 카디널리티 추정(UV 카운트)	수십 KB로 수억 추정, 오차 ~2% (Redis PFCOUNT)
Count-Min Sketch	빈도 추정(핫키 탐지)	과대추정 가능, 메모리 고정

G) 순서 유지 · 저장 엔진 레벨

구조	핵심	백엔드 등장
Skip List	확률적 다층 링크드 리스트, $O(\log n)$, 균형트리보다 구현 단순·동시성 유리	Redis Sorted Set(ZSet), 일부 인메모리 DB
LSM-Tree	쓰기를 메모리(MemTable)에 모아 순차 flush + 컴팩션 → 쓰기 최적화	Cassandra, RocksDB, LevelDB, Kafka 로그
B+ Tree vs LSM	B+ Tree=읽기·범위 강함/제자리 갱신, LSM=쓰기 폭주·압축 강함	워크로드에 따라 DB 엔진 선택
Inverted Index(역색인)	단어 → 문서ID 목록	Elasticsearch(이 프로젝트 정산 검색), 전문 검색

H) 캐시 교체 정책의 자료구조

- **LRU**: **HashMap**($O(1)$ 조회) + **이중 연결 리스트**($O(1)$ 최근성 갱신) 조합. 가장 오래 안 쓴 항목 제거.
- **LFU**: 빈도 카운트 기반, 자주 쓰는 것 보존(빈도 버킷 + 리스트).
- **Window-TinyLFU**: LRU의 최신성 + LFU의 빈도를 결합 — **Caffeine(4장 L1 캐시)**의 제거 정책. 확률적 빈도 스케치로 적은 메모리에 높은 적중률.

I) 한눈에 — "이럴 땐 이 구조"

요구사항	자료구조
키로 $O(1)$ 조회	해시 테이블
정렬 유지 + 범위 질의	균형트리 / B+Tree / Skip List
디스크 인덱스(읽기·범위)	B+Tree
쓰기 폭주 저장	LSM-Tree
가장 급한 것 먼저	힙 / 우선순위 큐
접두사 검색·자동완성	트라이
FIFO 작업 처리	큐 (메모리 → 분산 → DB-backed)
"없음" 빠른 판단·캐시 보호	블룸 필터

요구사항	자료구조
고유 수 추정(대용량)	HyperLogLog
최근/자주 쓰는 것 유지	LRU/LFU(HashMap+DLL)
의존성 순서	그래프 + 위상정렬
전문 검색	역색인

설계·면접 포인트: 자료구조 선택은 접근 패턴(읽기 vs 쓰기 비율, 점 조회 vs 범위, 정렬 필요 여부)과 제약(메모리, 디스크 I/O, 동시성)의 함수다. "왜 DB 인덱스는 해시가 아니라 B+Tree인가?"(범위 스캔·정렬 때문) 같은 질문에 트레이드오프로 답할 수 있어야 한다.

부록2) 코딩테스트 필수 자료구조와 적용

부록1이 "시스템 설계 관점"이라면, 이 장은 **알고리즘 문제풀이(코테) 관점**이다. 코테는 문제의 신호를 보고 맞는 자료구조·패턴을 떠올리는 게임이다. 핵심은 (1) 입력 크기로 목표 복잡도 추정 → (2) 신호 매칭 → (3) 적합한 구조 적용.

A) 입력 크기 → 목표 복잡도 (먼저 계산)

대략 1초 $\approx$ 1억(10^8) 연산 기준.

n (입력 크기)	허용 복잡도	떠올릴 접근
$n \leq 10 \sim 12$	$O(n!)$, $O(2^n)$	순열·완전탐색·비트마스크
$n \leq 20 \sim 25$	$O(2^n)$	부분집합·비트마스크 DP
$n \leq 500$	$O(n^3)$	플로이드, 3중 DP
$n \leq 5,000$	$O(n^2)$	2중 루프 DP
$n \leq 10^5 \sim 10^6$	$O(n \log n)$	정렬, 힙, 이분탐색, 세그트리
$n \leq 10^7 \sim 10^8$	$O(n)$, $O(n \log n)$	투포인터, 누적합, 해시
n 매우 큼	$O(\log n)$, $O(1)$	이분탐색, 수식, 분할정복

먼저 "n이 이만하니 $O(?)$ 여야 한다"를 정하면 자료구조가 거의 결정된다. 예: $n=10^6$ 인데 $O(n^2)$ 떠오르면 틀린 방향.

B) 패턴 ↔ 자료구조 (코테 빈출 매핑)

문제 신호(키워드)	패턴	자료구조	Java
"정렬된 배열에서 쌍/구간"	투 포인터	배열	<code>int[]</code> , 정렬
"연속 부분배열/윈도우 합·길이"	슬라이딩 윈도우	배열 + 두 인덱스	<code>int[]</code>
"구간 합을 여러 번"	누적합(Prefix Sum)	배열	<code>long[]</code> prefix
"직전 큰/작은 값", "온도·히스토그램"	모노토닉 스택	스택	<code>ArrayDeque</code>

문제 신호(키워드)	패턴	자료구조	Java
"괄호 짝/후위표기/되돌리기"	스택(LIFO)	스택	<code>ArrayDeque</code>
"최단 거리(가중치 1)/레벨 탐색"	BFS	큐	<code>ArrayDeque</code>
"윈도우 내 최대/최소"	모노토닉 덱	덱	<code>ArrayDeque</code>
"K번째/상위 K개/최솟값 반복 추출"	힙	우선순위 큐	<code>PriorityQueue</code>
"스트림 중앙값"	두 힙(max+min)	힙 2개	<code>PriorityQueue</code> ×2
"보수·중복·빈도·존재여부"	해싱	해시맵/셋	<code>HashMap</code> , <code>HashSet</code>
"정렬 유지 + 범위/이웃 조회"	균형트리	정렬맵/셋	<code>TreeMap</code> , <code>TreeSet</code>
"그룹·연결·사이클 합치기"	유니온-파인드(DSU)	서로소 집합	<code>int[] parent</code>
"선후 관계·의존성 순서"	위상정렬	그래프+진입차수	<code>List[]</code> + 큐
"가중치 최단경로(≥ 0)"	다익스트라	그래프+힙	<code>PriorityQueue</code>
"음수 간선/사이클"	벨만-포드·플로이드	거리 배열	<code>int[][]</code>
"최소 연결 비용(MST)"	크루스칼/프림	DSU/힙	<code>PriorityQueue</code> +DSU
"접두사·사전·문자열 집합"	트라이	트라이	<code>Map<Character, Node></code>
"구간 합/최댓값 갱신 찾기"	세그먼트 트리 / 펜윅(BIT)	트리 배열	<code>long[] tree</code>
"겹치는 부분문제·최적화"	DP(메모이제이션)	테이블	<code>int[][] dp</code>
"범위 안에서 답 탐색"	파라메트릭 서치	이분탐색	<code>lo/hi</code>

C) 코테 빈출 자료구조 — 적용 포인트

C-1. 스택 / 큐 / 덱 — `ArrayDeque` 하나로

- Java 코테에서 **스택도 큐도 덱도 `ArrayDeque`** 가 정석(`Stack/LinkedList`보다 빠름).
- 모노토닉 스택**: 증가/감소 단조성을 유지하며 "다음 큰 수/이전 작은 수"를 $O(n)$ 에 — 히스토그램 최대 직사각형, 일일 온도.
- 모노토닉 덱**: 슬라이딩 윈도우 최댓값을 $O(n)$ 에.

C-2. 우선순위 큐(힙) — `PriorityQueue`

- Top-K(`new PriorityQueue<>(Comparator)`), 다익스트라, 작업 병합(Merge K Lists), 중앙값(두 힙).
- 최대 힙은 `Collections.reverseOrder()` 또는 $(a,b) \rightarrow b-a$.

C-3. 해시 — `HashMap` / `HashSet`

- Two Sum**(보수 `target-x` 조회), 빈도수(`merge(k,1,Integer::sum)`), 중복 제거, 구간합+해시(부분합 K개).
- 순서 필요하면 `LinkedHashMap`, 정렬·이웃(`ceiling/floor`) 필요하면 `TreeMap`.

C-4. 유니온-파인드(Disjoint Set, DSU)

- "같은 그룹인가? 연결되나? 사이클 생기나?" → **경로 압축 + union by rank** 로 사실상 $O(\alpha(n)) \approx O(1)$.
- 크루스칼 MST, 네트워크 연결성, 섬 개수(동적), 집합 병합.


```
int find(int x){ return parent[x]==x ? x : (parent[x]=find(parent[x])); } // 경로 압축
void union(int a,int b){ parent[find(a)]=find(b); }
```

C-5. 그래프 표현·순회

- 인접 리스트 `List<Integer>[] adj` 가 코테 기본(희소). BFS=`ArrayDeque`, DFS=재귀/스택.
- 위상정렬(Kahn): 진입차수 0인 노드를 큐에 → 의존성/선수과목/빌드 순서.
- 격자(그리드) 문제는 2D 배열이 곧 그래프(상하좌우 `dx/dy`).

C-6. 트라이(Trie)

- 단어 검색/자동완성/접두사 카운트, 비트 트라이(XOR 최대쌍).

C-7. 세그먼트 트리 / 펜윅 트리(BIT)

- "값이 계속 바뀌는데 구간 합/최솟값을 빠르게" → 갱신·질의 모두 $O(\log n)$.
- 펜윅(BIT)은 구간 합 전용으로 구현 짧음, 세그트리는 더 일반적(최댓값·lazy propagation).

C-8. DP 테이블 (자료구조로서의 메모이제이션)

- 1D/2D 배열이 곧 자료구조. 점화식 + 방문표(`dp[]`, `visited[]`)로 지수→다항 시간.
- 비트마스크 DP(`dp[1<n]`)는 작은 n 의 집합 상태 압축.

D) "이 신호 보이면 이거" 빠른 치트시트

문제에 이런 말이 나오면	심중팔구
"가장 가까운 / 다음으로 큰"	모노토닉 스택
"최단 / 최소 횟수 (가중치 없음)"	BFS
"K번째 / 상위 K / 매번 최솟값"	힙
"정렬되어 있다 / 두 수의 합"	투 포인터·이분탐색
"연속 구간 / 윈도우"	슬라이딩 윈도우·누적합
"그룹 / 연결 / 친구의 친구"	유니온-파인드
"순서를 정해야 / 선행조건"	위상정렬
"경우의 수 / 최댓값(겹치는 선택)"	DP
"접두사 / 사전"	트라이
"범위를 정해 답을 맞춘다"	파라메트릭 서치

실전 순서: ① n 보고 목표 복잡도 → ② 신호로 패턴 후보 좁히기 → ③ 자료구조 확정 → ④ 엣지케이스(빈 입력·중복·음수·오버플로 `long`) 점검. 자료구조는 1~5장처럼 **실무에서도 같은 원리로** 쓰인다(코테 해시 = 실무 인덱스, 코테 큐 = 실무 메시지 큐).

부록3) 자바의 자료구조 — 타 언어와 비교

이 프로젝트는 Java 25 기반이다. 자바 컬렉션을 **Python · C++(STL) · JavaScript · Go** 와 비교하면, 자바의 설계 선택 (인터페이스/구현 분리, 제네릭+오토박싱, 풍부한 동시성 컬렉션)이 선명해진다.

A) 개념 ↔ 언어별 대응표

개념	Java	Python	C++ (STL)	JavaScript	Go
동적 배열	ArrayList	list	vector	Array	slice
연결 리스트	LinkedList	deque	list	—	container/list
해시 맵	HashMap	dict	unordered_map	Map / {}	map
정렬 맵	TreeMap(RB트리)	— (sortedcontainers)	map(RB트리)	—	—
해시 셋	HashSet	set	unordered_set	Set	map[T]struct{}
정렬 셋	TreeSet	—	set	—	—
스택 (LIFO)	ArrayDeque	list	stack	Array push/pop	slice
큐 (FIFO)	ArrayDeque	collections.deque	queue	Array push/shift	slice / chan
덱	ArrayDeque	collections.deque	deque	—	—
힙/우선순위큐	PriorityQueue	heapq	priority_queue	—	container/heap
삽입순서 맵	LinkedHashMap	dict(3.7+ 기본)	—	Map(기본)	—
불변 값객체	record	tuple/namedtuple	struct/tuple	— (Object.freeze)	struct

B) 자바만의 특징

B-1. 인터페이스 / 구현 분리

- 자바는 자료구조를 인터페이스(List, Set, Map, Queue, Deque)로 추상화 하고 구현체(ArrayList, HashMap ...)를 갈아끼운다.
- List<X> xs = new ArrayList<>() 처럼 변수는 인터페이스 타입 으로 — 구현 교체가 쉽다. Python/JS엔 이런 분리가 없다(타입 자체가 곧 구현).

B-2. 제네릭 + 오토박싱 — 자바의 아픈 점

- 자바 제네릭은 참조 타입만 담는다 → List<Integer> 는 int 가 아니라 박싱된 Integer 객체(헤더+포인터)를 담는다.
- 결과: int[] 대비 메모리 多·캐시 지역성 ↓·오토박싱 비용. 코테에서 성능이 뽀뽀하면 int[]/long[] 원시 배열을 쓴다.
- C++ vector<int> 는 진짜 연속 int → 가장 빠름. Python list 도 사실 객체 포인터 배열이라 느리지만 유연.
- 대안: fastutil·Eclipse Collections(원시 컬렉션), 미래엔 Project Valhalla(value types).

B-3. null · 순서 · fail-fast

- HashMap: null 키 1개 허용, 순서 보장 없음. LinkedHashMap: 삽입 순서. TreeMap: 키 정렬, null 키 불가.
- ↔ Python dict 는 3.7+ 부터 삽입 순서 보장(자바 LinkedHashMap 에 해당). 자바 HashMap 에 순서를 기대하면 버그.
- 자바 컬렉션은 fail-fast 반복자 — 순회 중 구조 변경 시 ConcurrentModificationException.

B-4. 동시성 컬렉션이 풍부 (자바의 강점)

- ConcurrentHashMap, CopyOnWriteArrayList, BlockingQueue, ConcurrentLinkedQueue 등 표준 제공 — 이 프로젝트 RateLimitRegistry(2장)가 실제 사용.
- ↔ Go: map 은 동시 쓰기 비안전 → sync.Map/뮤텍스, 대신 채널(channel) 이 일급 동시성 큐. Python: GIL로 사실상 단일코어, 일부 연산만 원자적. C++: 표준 동시 컨테이너 빈약(외부 TBB 등).

C) 언어 갈아탈 때 헛갈리는 함정

함정	설명
C++ map vs unordered_map	C++ map=정렬(RB트리, O(log n)), unordered_map=해시(O(1)). 자바 HashMap≈unordered_map, TreeMap≈C++ map. 이름만 보고 헛갈리지 말 것
Python엔 내장 정렬맵/셋 없음	자바 TreeMap/TreeSet, C++ map/set 은 내장. Python은 bisect 또는 sortedcontainers(서드파티)
dict 순서	Python 3.7+ 삽입순서 보장 ≠ 자바 HashMap(무순서). 자바에서 순서 필요하면 LinkedHashMap
Go map 순회 랜덤	Go는 맵 순회 순서를 의도적으로 랜덤화(순서 의존 코드 방지). 정렬하려면 키 뽑아 sort
JS Object vs Map	Object 키는 문자열/심볼만, Map 은 임의 키+삽입순서. 힙 정렬맵은 JS 내장에 없음
스택 클래스 함정(자바)	자바 java.util.Stack 은 레거시(동기화·느림) → ArrayDeque 사용 권장. LinkedList 보다도 ArrayDeque 가 스택·큐로 빠름

D) 성능·메모리 한눈에

항목	Java	C++	Python
원시 배열 합산	<code>int[]</code> 빠름	<code>vector<int></code> 최速	<code>list</code> 느림(객체 포인터)
제네릭 컬렉션	박싱 오버헤드(<code>Integer</code>)	템플릿=값 그대로	전부 객체
정렬맵 구현	RB트리(<code>TreeMap</code>)	RB트리(<code>map</code>)	내장 없음
동시성	표준 컬렉션 풍부	빈약	GIL 의존

요약: 자바는 _인터페이스 추상화_와 *동시성 컬렉션* 이 강점이고, *오토박싱* 이 약점이다. 알고리즘 본질(해시 = $O(1)$, 트리 = $O(\log n)$)은 언어 불문 동일하니, 개념은 1~부록2장에서 익히고 언어별 API·함정만 위 표로 매핑 하면 된다.